

Перевод от DeepSeek'a, необходимо проверять на соответствие с действительностью с оригиналом!

4COM1042 [Вычислительные платформы] Совместный групповой проект Б

"Игра в TV-теннис"

Заметки Алекса Шафаренко и Стива Ханта, 2016

1. Обзор

В рамках этого проекта вы реализуете игру "TV-теннис".

Вам предстоит создать видеозэкран с разрешением 32x32, а также кинематический контроллер — чип, который будет перемещать ракетки и мяч по экрану. Для отображения текущего счёта двух игроков вы будете использовать набор семисегментных индикаторов. Также вам нужно будет написать программу, которая предсказывает траекторию движения мяча и позиционирует ракетку компьютера так, чтобы отбить его.

TV-теннис — одно из названий первой в истории видеоигры (игра Pong). Посмотрите следующие видео:

- <https://youtu.be/XNRx5hc4gYc> — изобретатель Ральф Бэр играет в неё
- <https://youtu.be/-l73oK9q-jk> — история игры от Смитсоновского института

В этом проекте мы попытаемся воспроизвести некоторые особенности TV-тенниса, а также заставить компьютер играть с человеком максимально увлекательно.

Конечно, мы вооружены нашими превосходными знаниями в области вычислений по сравнению с прототипом 1960-х годов, но нас ограничивает довольно примитивная технология симуляции Logisim, которая не позволяет нам использовать несколько контроллеров ракеток одновременно и ограничивает разрешение видео до 32x32.

Тем не менее, вполне возможно добиться реалистичного игрового процесса; более того, поскольку симуляция процессора также довольно базовая (и медленная), необычайно легко создать систему, в

которой компьютер и человек-игрок будут на равных. Этому также способствует неудобство симулированного джойстика: подвести ракетку к нужному месту на доске непросто, особенно если джойстик вышел за пределы датчика.

2. Роботизированный игрок

Новый элемент в нашей игре по сравнению с классической версией — это роботизированный игрок: компьютер, который управляет одной из ракеток.

Компьютер всегда играет за правую ракетку, в то время как левой ракеткой управляет человек с помощью симулированного джойстика. Чтобы немного уравнять игру, мы ограничили роботизированного игрока:

Компьютер может двигать свою ракетку только тогда, когда мяч находится в левой половине экрана (половине игрока-человека). Как только мяч попадает в правую половину, игровой контроллер (аппаратный блок) блокирует правую ракетку.

Таким образом, компьютер должен сделать свои прогнозы и переместить правую ракетку до того, как мяч окажется на его половине экрана. Поскольку симуляция происходит на уровне логических вентилей, компьютер работает медленно. Иногда он не будет успевать достаточно быстро и пропустит мяч.

3. Возможные расширения

Несмотря на то, что мы ограничили компьютер, он всё равно лучше оснащён для прогнозирования, чем человек, и это может привести к скучной игре. Чтобы сделать игру интереснее, в нашей эталонной реализации были введены две дополнительные функции, которые вы можете рассмотреть как расширения базовой системы после того, как она заработает:

1. ****Диффузия.**** Чтобы добавить интереса, контроллер применяет силу диффузии по вертикали, т.е. случайным образом немного сбивает мяч с курса вверх или вниз. Таким образом, траекторию нельзя предсказать, просто построив прямую линию; вместо этого нужно определить узкий диапазон возможных направлений, в которых может полететь мяч. Это означает, что программа должна работать методом "счисления пути" (dead reckoning). Иногда, когда расчёт неверно предсказывает направление отскока мяча, компьютеру может потребоваться изменить положение ракетки более одного раза во время полёта мяча. В ситуации, когда угол падения очень острый, программе нужно проделать больше работы, чтобы учесть прогнозируемые отражения, и человек может гораздо легче выиграть очко.

2. ****Передача скорости.**** Более того, эталонная система включает в себя схему передачи скорости, которая передаёт часть импульса ракетки мячу, заставляя мяч "крутиться" при слёте с ракетки. Это изменяет угол отражения и даёт человеку возможность, хотя и довольно сложную, атаковать робота кручёными ударами под острым углом.

Естественно, от вас не ожидают разработки ни одной из этих расширенных функций, но полезно знать, что их можно добавить в рамках ограничений данной технологии симуляции и процессорного ядра.

Вы не ограничены в том, как расширять систему, как только она заработает. Вы можете изменить её функциональность или добавить другие дополнительные функции по своему усмотрению.

4. Проектирование аппаратного обеспечения

Этот проект в основном касается аппаратной части. Игровая панель состоит из нескольких компонентов, соединённых довольно простым образом. Один из этих компонентов — кинематический контроллер, обозначенный 'C' на диаграмме, — является довольно сложным. Мы отложим рассмотрение этого компонента напоследок и сначала рассмотрим другие.

[Здесь была вставка изображения схемы]

5. Панель дисплея, ракетки и мяч

У нас есть универсальная панель дисплея, состоящая из 1024 пикселей, расположенных в 32 столбца по 32 пикселя в каждом. Столбцы нумеруются от 0 до 31 слева направо, а пиксели в каждом столбце нумеруются от 0 до 31 снизу вверх.

Каждый столбец имеет 32-битный входной контакт, по одному биту на пиксель. Рисунок, отображаемый столбцом, определяется 32-битным шаблоном, который подаётся на этот контакт (единицы в шаблоне включают пиксели, нули — выключают). При правильных входных сигналах эта панель может отображать любой рисунок 32x32 пикселя.

Для TV-тенниса нам нужно отображать 2 ракетки и мяч на однотонном фоне. Мяч рисуется как один пиксель, а каждая ракетка рисуется как вертикальная линия из 3 смежных пикселей в одном столбце. Мяч может появляться в любом месте экрана, но ракетки могут появляться только в фиксированных столбцах (левая ракетка в столбце 3, правая ракетка в столбце 28).

Чтобы отобразить мяч, нам нужно включить 1 пиксель из 32 в определённом столбце на экране. Нам нужно знать столбец (координата x) и строку (координата y), в которых должен появиться мяч. Чтобы отобразить одну ракетку, мы включаем 3 смежных пикселя из 32 в фиксированном столбце, поэтому нам просто нужно указать строку (координату y), в которой должен находиться нижний конец ракетки. Таким образом, нам нужны четыре 5-битных числа, чтобы задать положения мяча ($ballX$, $ballY$) и двух ракеток ($leftbatY$, $rightbatY$). Они будут поступать от кинематического контроллера.

6. Видеоподсистема

Видеоподсистема имеет четыре 5-битных входа: $ballX$, $ballY$, $leftbatY$, $rightbatY$. Панель дисплея управляется набором из 32 видеочипов, и все эти четыре входа подключены к каждому видеочипу. Однако, чтобы уменьшить количество проводки, видеочипы соединены в цепочку. Чипы расположены в цепочке с запада на восток; самый западный чип получает все входные данные и передаёт их следующему чипу в цепочке.

Вместо того чтобы соединять в цепочку 32 отдельных видеочипа, мы используем ячеистую конструкцию. 8 чипов соединяются в цепочку, образуя видео-секцию. Затем 4 такие секции соединяются в цепочку. Ячеистая конструкция — это стандартный приём: спроектировать одну ячейку, добавить её в библиотеку схем, а затем собрать более крупную схему, соединив вместе несколько идентичных ячеек по регулярному шаблону. Каждая секция обозначена на принципиальной схеме прямоугольником, содержащим диагональный крест.

7. Видеочип

Каждый видеочип имеет входные контакты $ballX$, $ballY$, $leftbatY$, $rightbatY$ на западной стороне и выходные контакты $ballX$, $ballY$, $leftbatY$, $rightbatY$ на восточной стороне, и битовые шаблоны на каждом из них проходят через каждый видеочип без изменений.

Каждый видеочип управляет конкретным столбцом панели дисплея, и каждому чипу нужно знать, каким столбцом он управляет, поэтому чипы нумеруются от 0 до 31, как и столбцы. Вместо того чтобы хранить разный номер столбца в каждом чипе индивидуально, мы используем небольшой трюк.

Мы даём каждому видеочипу дополнительный 5-битный входной контакт с меткой $chipID$ на западной стороне и добавляем соответствующий выходной контакт на восточной стороне, но на этот раз мы добавляем 1 к числу перед его повторной выдачей. Затем, когда мы соединяем чипы в цепочку, мы подаём $0b00000$ на вход $chipID$ самого западного чипа (= столбец 0), который добавляет

1 и, таким образом, отправляет сигнал 0b00001 следующему чипу в цепочке. Таким образом, все 32 чипа нумеруются автоматически.

| Входы западной стороны | Выходы восточной стороны |

| --- | --- |

| chipID | chipID+1 |

| ballX | ballX (без изменений) |

| ballY | ballY (без изменений) |

| leftbatY | leftbatY (без изменений) |

| rightbatY | rightbatY (без изменений) |

Важно понимать, что видеочип является чисто комбинационным. Видеочип не имеет внутреннего состояния и создаёт 32-битный выходной шаблон, который управляет одним столбцом панели дисплея. Этот шаблон выводится на единственный северный контакт. Итак, как создаётся этот шаблон?

Каждый чип содержит схему для отображения мяча и схему для отображения одной ракетки.

Позиция мяча получается на входных контактах ballX и ballY. Каждый чип сравнивает свой chipID с ballX, и если они равны, он зажигает пиксель под номером ballY в своём столбце дисплея. Это простая комбинационная логика.

Поскольку ракетки находятся в фиксированных столбцах, мы можем "защитить" номера их столбцов (3 и 28) в видеоподсистему. Каждый видеочип содержит две 5-битные константы: 0b00011 и 0b11100. Эти две константы сравниваются с chipID, так что видеочип, у которого chipID равен 00011, отображает левую ракетку с её нижним пикселем в строке leftbatY, а видеочип, у которого chipID равен 11100, отображает правую ракетку с её нижним пикселем в строке rightbatY. Это также достигается с помощью комбинационной логики.

****ПРИМЕЧАНИЕ:**** Ни одному чипу никогда не потребуется отображать обе ракетки одновременно, но возможно, что мяч окажется в том же столбце, что и одна из ракеток, поэтому конструкция должна это допускать.

8. Кинематический контроллер

Задача кинематического контроллера — перемещать мяч, обновляя его координаты, блокировать правую ракетку, когда мяч находится в правой половине экрана, и сообщать положения мяча и двух ракеток другим частям системы (программе и видеоподсистеме).

Всё остальное делается другими аппаратными и программными частями: левая ракетка управляется непосредственно (симулированным) джойстиком, а правая ракетка управляется (через интерфейс ввода-вывода) ядром CdM-8, на котором выполняется программа робота-игрока.

Контроллер обновляет координаты и скорость мяча и сообщает их программе, которая использует эти значения для прогнозирования траектории полёта мяча и перемещения своей ракетки. Кинематическому контроллеру необходимо знать положение обеих ракеток, чтобы определить, не попал ли мяч в одну из них, а также он отвечает за "блокировку" правой ракетки, когда мяч находится в правой половине экрана.

9. Позиция мяча

Мяч движется в квадратном пространстве, которое разделено на ячейки сеткой 256×256, при этом ячейки нумеруются от 0 до 255 слева направо и от 0 до 255 снизу вверх, так что самая нижняя левая ячейка имеет координаты 0,0. Координаты x и y мяча хранятся как 8-битные беззнаковые числа, и все расчёты движения и обновления позиции выполняются с этими 8-битными значениями.

10. Управление дисплеем

Поскольку контроллер имеет позицию мяча и y -координаты двух ракеток, логично использовать его для передачи их в видеоподсистему для отображения.

К сожалению, панель дисплея не может показать местоположение мяча с такой высокой точностью, потому что она имеет размер всего 32 на 32 пикселя. Мы делаем так, чтобы каждый пиксель на дисплее соответствовал многим ячейкам сетки в пространстве (64 ячейки сетки на один пиксель). Например, пиксель в столбце 2 и строке 5 на экране будет использоваться для отображения мяча, когда его фактическая x -координата находится между 16 и 23, а его y -координата — между 40 и 47.

Преобразование координат контроллера в номера столбцов и строк экрана легко достигается отправкой только 5 самых старших битов x и y координат мяча в видеоподсистему. y -координаты ракеток повсюду хранятся как 5-битные значения, поэтому преобразование не требуется.

11. Представление скорости мяча

Скорость мяча, v_x , имеет горизонтальную (x) составляющую, v_x , и вертикальную (y) составляющую, v_y . Каждая из них представлена 3-битным числом в дополнительном коде (со знаком), что означает, что v_x и v_y могут принимать значения в диапазоне от -4 до +3. Это очень грубая шкала, но для наших целей её достаточно.

Положительная скорость x означает, что мяч движется слева направо по пространству, а положительная скорость y означает, что мяч движется снизу вверх. Отрицательные скорости, конечно, означают, что мяч движется в противоположном направлении. Любая комбинация v_x и v_y возможна.

Каждый элемент данных, считываемый ядром CdM-8, должен быть представлен как 8-битная строка, поскольку CdM-8 — это 8-битная платформа. Вместо того чтобы дополнять каждую из двух компонентов скорости до 8 бит, мы объединяем их в одну 8-битную строку следующим образом: биты 0-2 содержат v_x , биты 3-5 содержат v_y , а биты 6-7 всегда установлены в 0.

12. Отражение мяча

Когда мяч ударяется о ракетку или достигает одного из вертикальных краёв пространства, он отскакивает (отражается) путём отрицания v_x . Когда он ударяется о верхний или нижний край пространства, он отражается путём отрицания v_y . Только когда мяч попадает точно в угол, и v_x , и v_y должны быть отрицаны, чтобы он отскочил.

Следует отметить, что -4 не может быть правильно преобразовано в положительное число, так как не существует 3-битного представления в дополнительном коде для +4. Фактически, 3-битное дополнительное значение -4 также равно -4, поэтому скорость мяча в этом направлении вообще не изменится. В этом случае отражённое значение скорости устанавливается равным +3.

13. Внутренняя память (регистры)

Для начала контроллеру нужны две пары регистров: пара 8-битных для x и y координат мяча и пара 3-битных для x и y компонентов его скорости.

Он также включает два 8-битных регистра для хранения счёта двух игроков и 5-битный регистр, в который защёлкивается положение правой ракетки. Последний регистр используется для реализации блокировки перемещения ракетки компьютером, когда мяч находится в правой половине экрана. Новое положение правой ракетки может быть защёлкнута в регистр только тогда, когда мяч находится в столбце дисплея, номер которого не превышает 15.

Это всё состояние, которым нужно управлять. Остальная часть схемы является чисто комбинационной.

14. Расчёт движения

Траектория мяча вычисляется итеративно, шаг за шагом, с использованием локального тактового сигнала. Данный тактовый сигнал не связан с тактовым сигналом процессора, даже если симуляция Logisim объединяет все тактовые сигналы, используемые в проекте. В некотором смысле наличие локального тактового сигнала означает, что контроллер сам по себе является крошечным компьютером, хотя у него нет памяти и он вычисляет исключительно с помощью регистров. У него также нет программы как таковой, поскольку он повторяет одну и ту же жёстко заданную "инструкцию" на каждом такте.

Вычисление представляет собой двухфазный процесс:

- **Фаза 1:** шаг движения мяча на основе текущей скорости.
- **Фаза 2:** изменение скорости на основе столкновений, обнаруженных в Фазе 1.

Фазу 1 легко вычислить:

$$x = x + vx$$

$$y = y + vy$$

Координаты x и y используются для обращения к ячейкам виртуального пространства движения мяча. Компоненты скорости x и y измеряются в единицах "ячеек, пройденных за такт", представленных как 8-битные строки, со знаковым расширением влево, как и должно быть. Новое положение мяча затем может быть вычислено с использованием двух 8-битных сумматоров.

15. Обнаружение момента, когда мяч достигает края

Очень просто обнаружить, когда мяч столкнулся с одним из краёв пространства движения: когда $vx < 0$, его прибавление к координате x должно дать перенос (равный 1); если этого не происходит, мяч только что ударился о левую стену. Когда $vx > 0$, его прибавление к координате x не должно давать перенос; если перенос происходит, мяч только что ударился о правую стену. Аналогично, при прибавлении vy к y мы можем использовать вывод переноса, чтобы определить, столкнулся ли мяч с потолком или полом.

На самом деле кинематический контроллер не обнаруживает столкновение с краем. Что он на самом деле обнаруживает, так это то, что мяч оказался бы за пределами пространства движения, если бы его позиция была обновлена на требуемую величину. Это, очевидно, недопустимо, поэтому, когда это обнаружено, новая координата не защёлкивается в регистре координат. Вместо этого скорость мяча в этом измерении отрицается, чтобы он "отскочил".

Обратите внимание, что есть два вида столкновений: горизонтальные и вертикальные. В столкновении участвует только одна координата, другая не участвует. Конечно, если мяч послан прямо в угол, он может удариться об обе стены одновременно; тем не менее, каждое направление столкновения будет обрабатываться независимо.

В Фазе 2 происходит обновление скорости. Если не было столкновения ни в горизонтальном (x), ни в вертикальном (y) измерении, скорость мяча остаётся прежней. Если было столкновение, скорость должна быть отрицана в этом измерении, изменив направление движения на противоположное.

Как нам реализовать вышеописанный двухфазный процесс? Простое решение — использовать обе половины тактового цикла. Когда тактовый сигнал высок, координаты пересчитываются на основе текущей скорости; когда тактовый сигнал низок, скорость отражается в зависимости от того, привело ли движение к столкновению. Это означает, что защёлки координат должны срабатывать по спадающему фронту тактового сигнала, а защёлки скорости — по нарастающему.

16. Отскок мяча от ракетки

Ракетки могут влиять только на горизонтальные отражения, так как они сами вертикальны. Процедура отражения почти точно такая же, как и для отражения мяча от стены. В последнем случае мы предполагаем, что стены находятся в точках $x = 0$ и $x = 255$, и используем перенос для обнаружения, когда мяч пытается пересечь эту границу.

При обнаружении контакта с ракеткой мы выполняем немного другой расчёт, но используем ту же технику обнаружения, исследуя выходной перенос. Левая ракетка находится в столбце 3 на дисплее, что означает, что она охватывает x -координаты в диапазоне 24..31. Если мы предположим, что мяч ударит в середину ракетки, это означает, что в точке контакта его x -координата будет равна 28.

Теперь, когда мы используем отрицательный x для обновления позиции мяча (т.е. он движется справа налево), мы сначала вычитаем 28 из старого значения x , затем прибавляем v_x , и если есть перенос, равный 0, мяч, возможно, ударил ракетку (но только если y -координата мяча помещает его в пределах длины ракетки).

-28 в 8-битном дополнительном коде — это $0xe4$. Расстояние для правой ракетки такое же, 28 единиц, только теперь мяч движется в противоположном направлении, поэтому расстояние положительное: $0x1c$.

17. Подсчёт очков в игре

Счёт игрока в игре — это количество отражений мяча от вертикальной стены на половине другого игрока. Контроллер включает два счётчика для ведения счёта. (В данном случае каждый счётчик представляет собой регистр с инкрементом в цепи обратной связи: мы используем для этого стандартный компонент Logisim).

Триггером для счётчиков очков служит сумматор, реализующий $x + vx$ в Фазе 1, а точнее — его схема обнаружения столкновения со стеной. Каждый раз, когда обнаруживается столкновение со стеной (напомним, что это делается путём обнаружения того, что перенос из сумматора противоположен знаку vx), это действует как сигнал разрешения для обновления счётчика очков в следующем (низкий уровень тактового сигнала) цикле Фазы 2. Какой игрок получает очко, зависит исключительно от знака vx в момент столкновения.

18. Краткое описание проекта

Подводя итог, можно сказать, что проект кинематического контроллера состоит из шести несложных частей схемы:

- Обновление X
- Обновление Y
- Отражение vx
- Отражение vy
- Обнаружение столкновения с ракеткой
- Ведение счёта

Связь между вышеуказанными частями следующая: обновления X и Y управляют отражениями vx и vy , причём на отражение vx также влияет обнаружение столкновения с ракеткой. Наконец, ведение счёта полностью находится под управлением обновления X .

Спецификации кинематического контроллера и двух вспомогательных чипов, от которых он зависит, находятся в приложении в конце этого документа.

19. Сопряжение игры с платформой CdM-8

Программа, выполняющаяся на процессоре, должна знать координаты и скорость мяча, чтобы делать прогнозы. Когда она вычислит желаемое положение правой ракетки, его необходимо сделать доступным для кинематического контроллера.

Следовательно, интерфейс должен включать один регистр ввода-вывода, в который программа будет записывать у-координату правой ракетки. Это 5-битный регистр, который должен использовать старшие биты шины I/Odat.

Давайте подключим игровую панель к IO-3 (адрес = 0x3) для этой игры. Тот же адрес можно использовать для чтения упакованной скорости мяча vxu, как это делает контроллер, в 8-битном слове с двумя неиспользуемыми старшими битами. В заметках под названием "Работа с полной системой CdM-8" (доступны отдельно) объясняется, как разделять адреса ввода-вывода.

Ещё два регистра следует использовать, чтобы программа могла читать 8-битные x и y координаты мяча, например, IO-4 и IO-5. Ни скорость, ни координаты не требуют регистров в интерфейсе, поскольку контроллер защёлкивает их на каждом такте и подаёт на выходные контакты восточной стороны. Всё, что нужно сделать интерфейсу, — это декодировать адрес ввода-вывода/направление передачи и подать эти значения на шину I/Odat.

20. Программное обеспечение

Программа должна предсказать траекторию мяча, учитывая его текущее положение и скорость. Она должна определить точку, в которой мяч пересечёт 28-ю вертикаль дисплея (считая от 0), и записать её y-координату в регистр IO-3. Она должна делать это достаточно быстро, потому что если за это время мяч продвинется до средней точки дисплея, контроллер не скопирует содержимое IO-3 на свой восточный вывод, и в результате правая ракетка не сдвинется, и мяч, скорее всего, ударится о правую стену.

Необходимость работать быстро означает, что попытка имитировать аппаратный алгоритм в программном обеспечении не даст удовлетворительных результатов, поскольку процессор слишком медленный по сравнению с контроллером. Вместо того чтобы подражать контроллеру, добавляя небольшие приращения к координатам и вычисляя следующее местоположение, пока мы

не приблизимся к ракетке, мы должны вычислить конечное местоположение мяча в точке пересечения за один шаг.

****Расстояние между мячом и правой ракеткой по горизонтальной оси равно 224 - x**** (здесь и далее мы будем измерять любые расстояния в субпикселях, масштабируя поле до размера 256×256). Если горизонтальная скорость мяча равна vx, мячу потребуется $(224 - x) / vx$ тактов контроллера, чтобы достичь линии ракетки. Его вертикальное смещение за это время составит $(224 - x) / vx \times vy$. Это смещение, прибавленное к текущей y-координате, определяет точку пересечения на линии ракетки, в которую мяч прибудет. Или, скорее, прибыл бы, если бы мы могли гарантировать, что он не столкнётся с горизонтальной стеной.

[Здесь была вставка изображения]

В текущем проекте никакой другой объект, кроме мяча, никогда не движется в x-направлении; это означает, что величина vx постоянна. Контроллер устанавливает vx равным -2 (110 в 3-битном дополнительном коде) при сбросе, поэтому абсолютное значение горизонтальной скорости остаётся равным 2. С другой стороны, vy также является 3-битным числом, поэтому его абсолютное значение может быть не более 4.

Это означает, что мы можем легко как делить, так и умножать в программе. Мы делим на 2, выполняя сдвиг вправо, и умножаем на 2 и 4, выполняя сдвиги влево; наконец, умножение на 3 сводится к сдвигу и сложению.

В процессе "умножения" и сложения для получения смещения и конечной точки пересечения будут сохраняться только младшие 8 бит; нам нужно обращать внимание на бит переноса. Действительно, если результат d получен без переноса, равного 1, или с переносом, равным 1, возникающим чётное количество раз, d является y-координатой точки пересечения. Однако если бит переноса возник нечётное количество раз, d нужно вычесть из 256. (Но это просто 8-битный дополнительный код).

В приведённом выше анализе мы предположили, что вертикальная скорость vy положительна. Когда $vy < 0$, мало что меняется. Разница в том, что вместо подсчёта количества раз, когда был сгенерирован перенос 1 при выполнении расчёта, мы подсчитываем количество раз, когда произошёл перенос 0. Действительно, когда прибавление отрицательного к положительному даёт положительный результат, отрицательное число даст перенос 1 (поскольку оно уже имеет 1 в знаковом бите и может стать 0 только вместе с переносом 1).

Как минимум, программа должна уметь обрабатывать одно отражение. Это достаточно просто (требует лишь нескольких арифметических манипуляций и проверок) и совершенно достаточно,

если базовый контроллер не расширен для включения случайных флуктуаций скорости и/или передачи скорости от ракетки к мячу. Однако, если эти расширения будут предприняты, угол падения может стать очень острым, и в этом случае мяч может отскочить от стен дважды.

21. Предлагаемая последовательность разработки

1. ****Отладка программы.**** Используйте эмулятор сосоети для тщательной отладки вашего кода, прежде чем сопрягать его с игровой панелью. Вам следует использовать адреса ввода-вывода 0xf3 - 0xf5 как обычные адреса памяти и симулировать контроллер, помещая константы в них. Таким образом, можно полностью отладить программу прогнозирования траектории до того, как будут построены контроллер и другие схемы.

2. ****Видеоchip.**** Начинайте аппаратную работу параллельно с программной. Начните с построения одного чипа вертикальной строки, который отображает один столбец 32x32 дисплея. Чип должен выдавать 32-битную растровую картинку из нулей, или изображение мяча в один пиксель на определённой высоте, или изображение ракетки (3 последовательных пикселя на некоторой высоте), или и мяч, и ракетку. Всё будет зависеть от входных данных с восточной стороны. Используйте контакты для тщательной отладки вашей схемы. Затем соберите вместе 32 копии чипа и посмотрите, сможете ли вы управлять мячом и ракетками с помощью контактов. Вы готовы переходить к созданию контроллера.

3. ****Кинематический контроллер (базовый).**** Очень внимательно следуйте спецификации чипа контроллера. Всё много раз проверяйте. Сосредоточьтесь на защёлках X, Y, vx и vy и создайте работающий прототип, который сможет правильно перемещать мяч. Опустите части, отвечающие за столкновение с ракетками, и регистры счёта. Подключите контакты к контроллеру и задайте некоторые y-координаты для обеих ракеток. Нажимая на тактовый сигнал, проверьте, можете ли вы заставить мяч летать и отражаться. Затем подключите джойстик к входному контакту левой ракетки LEFTYIN, установите достаточно низкую частоту тактов и попробуйте поиграть против стен и неподвижной правой ракетки.

4. ****Интеграция.**** Если вы достигли этого этапа, ваш контроллер в основном работает. Быстро соберите интерфейс компьютера и попросите своего специалиста по программному обеспечению запустить игру. Однако разработайте свои собственные небольшие тесты, которые показывают, что правая ракетка правильно работает под управлением программы, чтобы, когда программист начнёт утверждать, что это аппаратное обеспечение не работает, у вас были убедительные доказательства обратного.

Приложение: Спецификации чипов

Чип 1: reflect

Следующие два чипа используются кинематическим контроллером. Каждый из них является чисто комбинационным. Другими словами, каждый из них действует так, как если бы он был математической или логической функцией.

Первый используется для изменения направления движения мяча либо в горизонтальном, либо в вертикальном направлении. Старая и новая скорости всегда противоположны по знаку, но чип не выполняет математически корректную смену знака, поскольку предполагает, что $-(-4) = 3$, что приводит к немного неправильному углу отражения мяча. Это лишь делает игру интереснее.

****Спецификация чипа****

****Имя:**** reflect

****Контакты ввода/вывода****

- Вход: oldv(3)

- Выход: newv(3)

****Комбинационная логика****

newv = если oldv = 0b100, то oldv, иначе oldv + 1

****Конец спецификации чипа****

Чип 2: inrange

Второй чип определяет, находится ли мяч в одной из строк экрана, занимаемых одной из ракеток. Это используется для определения того, контактируют ли ракетка и мяч.

****Спецификация чипа****

****Имя:**** inrange

****Контакты ввода/вывода****

- Вход: Yball(5), Ybat(5)

- Выход: shareArow

****Комбинационная логика****

shareArow = (Yball = Ybat) ИЛИ (Yball = (Ybat+1)) ИЛИ (Yball = (Ybat+2))

****Конец спецификации чипа****

Чип 3: kinematic_controller

****Спецификация чипа****

****Имя:** kinematic_controller**

****Использует комбинационные чипы:** reflect(3->3), inrange(5,5->1)**

****Контакты ввода/вывода****

- Вход: reset # Северная сторона
- # Вертикальные координаты ракеток (от джойстика и программы)
- Вход: leftYin(5) # Северная сторона
- Вход: rightYin(5) # Западная сторона
- # Координаты и скорость мяча (для программы CdM-8)
- Выход: xball(8), yball(8), vxu(8) # Западная сторона
- # scoreA, scoreB используются для обновления табло
- Выход: scoreA(8), scoreB(8) # Южная сторона
- # Номер первого столбца дисплея
- Выход: zero(5) # Восточная сторона
- # Координаты мяча (для дисплея)
- Выход: ballX(5), ballY(5) # Восточная сторона
- # Вертикальные координаты ракеток (для дисплея)
- Выход: rightYout(5), leftYout(5) # Восточная сторона

****Внутренние сигналы****

- Сигналы: vxneg, vyneg, xrefl, yrefl
- Сигналы: hitA, hitB # один из них будет истинным, когда мяч проходит мимо ракетки другого игрока
- Сигналы: Lbcontact, Rbcontact, Bcontact

- Сигналы: longVX(8), longVY(8)

****Защёлки (регистры)****

- register(8) xreg, yreg # хранят текущие координаты x и y мяча в пространстве 256x256

- register(3) vx, vy # x и y компоненты скорости мяча

- register(8) scA, scB # счёт двух игроков

- register(5) rightYreg # хранит y-координату правой ракетки

****Комбинационная логика****

- zero = 0b000000

- leftYout = leftYin # Положение левой ракетки передаётся без изменений

- rightYout = rightYreg # Положение правой ракетки берётся из регистра

- ballX = xreg[3:7] # 5 старших битов координат мяча

- ballY = yreg[3:7] # используются для определения местоположения мяча на экране

- vxy[0:2] = vx

- vxy[3:5] = vy

- vxy[6:7] = 0b00 # Скорость имеет 3-битную x компоненту и 3-битную y компоненту, дополнена до 8 бит двумя нулями

- # Расширяем каждую 3-битную скорость до 8-битного числа в дополнительном коде,
устанавливая 5 старших битов расширенной версии равными знаку 3-битной версии.

Необходимо, чтобы можно было прибавлять vx к x и vy к y

- longVX = знаковое_расширение(vx, с 3 до 8 бит, используя vx[2])

- longVY = знаковое_расширение(vy, с 3 до 8 бит, используя vy[2])

- vxneg = longVX[7] # Отрицательна ли скорость x?

- xrefl = (перенос_от(xreg + longVX)) ≠ vxneg # ударился ли мяч о стену?

- vyneg = longVY[7] # Отрицательна ли скорость y?

- yrefl = (перенос_от(yreg + longVY)) ≠ vyneg # ударился ли мяч о пол или потолок?

- hitA = xrefl ∧ ¬vxneg # заработал ли игрок A очко?

- hitB = xrefl ∧ vxneg # заработал ли игрок В очко?
- # Ударился ли мяч о левую ракетку?
- LBcontact = (((xreg + 0xe4) + longVX).carry ≠ vxneg) ∧ inrange(y, leftYin)
- # Ударился ли мяч о правую ракетку?
- RBcontact = (((xreg + 0x1c) + longVX).carry ≠ vxneg) ∧ inrange(y, rightYin)
- Bcontact = LBcontact ∨ RBcontact

****Поведение****

- ****по спадающему фронту тактового сигнала:****
 - xreg := # обновить x-координату мяча
 - если reset то 0xc0 # x в начале игры
 - иначе xreg + longVX # прибавить x-скорость к x
 - разрешено, когда: (¬xrefl ∧ ¬Bcontact) ∨ reset
 - yreg := # обновить y-координату мяча
 - если reset то 0x80 # y в начале игры
 - иначе yreg + longVY # прибавить y-скорость к y
 - разрешено, когда: ¬yrefl ∨ reset
 - # правая ракетка может двигаться только тогда, когда мяч не находится
 - # в правой половине экрана (номер столбца < 16):
 - rightYreg :=
 - если reset то 0x0f иначе RightYin
 - разрешено, когда: ¬ballX[4]
- ****по нарастающему фронту тактового сигнала:****
 - vx := # обновить x-скорость
 - если reset то 0b110 # изначально -2
 - иначе reflect(vx) # горизонтальный отскок
 - разрешено, когда: xrefl ∨ Bcontact ∨ reset
 - vy := # обновить y-скорость
 - если reset то 0b001 # изначально +1

- иначе $\text{reflect}(v_y)$ # вертикальный отскок
- разрешено, когда: $y_{\text{refl}} \vee \text{reset}$
- $\text{scA} := \text{scA} + 1$ разрешено по hitA
- $\text{scB} := \text{scB} + 1$ разрешено по hitB

****Конец спецификации чипа****